HW2: Distributed Matrix Multiplication with Cannon's Algorithm

CSCI 5451: Introduction to Parallel Computing

Due: November 3, 11:59 PM **Total: 30 points**

Overview

The assignment ZIP available on Canvas contains the materials for completing HW2, where you will implement and extend Cannon's algorithm for distributed matrix multiplication using MPI.

This assignment is worth **30 points**: **18 for the written component** (Questions 1–5) and **12 for programming**. Submit the entire written component in `report.pdf`.

1. Provided Materials

You are given the following files:

- `README_STUDENT.md` — The Markdown version of this assignment document, including the programming instructions, written questions, implementation analysis requirements, and grading.
- `square_cannon.c` — Starter C implementation of Cannon's algorithm for square matrices. This allows you to compute $C = AB$, given an input text file containing both A and B. This starter code can successfully compute Tests 0–4, given that correct numbers of processes are chosen (see `autograder.sh` for the process counts).
- `tests/` — Directory containing input/output files for all test cases (there are no hidden tests for this assignment). Test i may be found at `tests/test_`i. Each test directory contains `in.txt`, with both matrices A and B, and `out.txt`, with the expected matrix C.
- `autograder.sh` — Bash script to compile and test your code across all tests we will consider. This contains the exact set of commands we will run for each test.
- `autograde_runner.py` — Computes your autograded score from the logs of `autograder.sh`, using the number of speedup/unit tests your program passes.

These materials allow you to run Cannon's algorithm for **square matrices** where the number of rows/columns is divisible by the square root of the number of processes ($\sqrt{p}$). The provided code does not work when the rows/columns are not divisible by $\sqrt{p}$, or when the matrices A and B are not square.

2. Your Task

You must update the program to support **general matrix dimensions**. In all tests, you can assume that the number of rows/columns in A, B is at least $\sqrt{p}$, and that we will always launch a square number of processes up to 64: $1, 4, 9, 16, 25, 36, 49, 64$. To make your task easier, complete the following in order:

(a) Non-square matrices: Ensure your code works for matrices where the number of rows/columns is divisible by $\sqrt{p}$, even if the matrices are not square. These correspond to Tests 5–7 (see `autograder.sh` for the process counts).

(b) Arbitrary dimensions: Generalize your code to handle matrices of any size, even when dimensions are not divisible by $\sqrt{p}$. Tests 8–11 correspond to this case; Test 11 is used for speedup tests. Once you have completed this portion, you should be able to run Tests 8–11 with any square process count in the list above.

3. Getting Started

Before beginning the written questions, set up your working copy and run the square-matrix test below. Keep `square_cannon.c` available as a reference while you work through the questions and extend `variable_cannon.c`.

- **Ensure that you have downloaded the zip file for this assignment from canvas**
- **Create your implementation:** Copy the starter file to begin:

```
cp square_cannon.c variable_cannon.c
```

  You will edit `variable_cannon.c` for your solution.
- **Create a hostfile:** Make a file named `hosts.txt` listing the node you will use (e.g., your assigned plate node). The number of slots you should use in this file is 64.

4. Running a Single Test

To run a specific test manually (in this case, four processes on Test 0):

```
mkdir -p build
mpicc -O3 -std=c99 -o build/variable_cannon variable_cannon.c -lm
mpirun -np 4 --hostfile hosts.txt --map-by node \
  build/variable_cannon tests/test_0/in.txt build/out_test0_np4.txt
```

If successful, this will output a print statement to the shell like:

```
Time for matrix multiplication: %.6f seconds
```

where `%.6f` is replaced by a floating-point number.

Additionally, this will create a file named `build/out_test0_np4.txt`. Its contents should match those found in `tests/test_0/out.txt`.

This example uses Test 0. Replace the input file and output file as needed to check the performance of other tests. Preserve the timing-output format so that the grading scripts can read it.

5. Running the Full Autograder Script

To run all tests and see pass/fail results:

```
./autograder.sh
```

6. Running the Full Autograde Runner (for a Score)

To run the autograder and receive a scored grade in addition to the logs:

```
python3 autograde_runner.py
```

7. Written Questions

When appropriate, use the starter code as a reference, and justify your answers using the mathematics of matrix multiplication. Use zero-based indexing. Process $P_{i,j}$ denotes the process at coordinates (i, j) in the process grid.

Question 1: Why Square Cannon Works (4 points)

Let A and B be 12×12 matrices distributed over a 3×3 process grid. Process $P_{i,j}$ initially holds blocks $A_{i,j}$ and $B_{i,j}$ and computes $C_{i,j}$.

Initially, row i of A shifts left by i positions, and column j of B shifts upward by j positions. After each multiplication stage, A shifts one position left and B one position upward. All shifts wrap around.

(a) Trace one process. [2 points]

Complete the table for $P_{1,2}$. Stage 0 is immediately after the initial alignment.

Stage	Block of A held	Block of B held	Contribution to $C_{1,2}$
0			
1			
2			

(b) Explain the algorithm. [2 points]

For a general $q \times q$ process grid, derive which blocks process $P_{i,j}$ holds at stage s, where $0 \leq s < q$. Use your expressions to explain why the initial alignment and q multiplication stages compute $C_{i,j}$ correctly.

Question 2: Rectangular Blocks (4 points)

Let A be 12×18 and B be 18×24, using the same 3×3 process grid.

(a) Determine the block sizes. [2 points]

Give the dimensions of each local A, B, and C block, and the number of scalar elements in each A-block and B-block exchange. Generalize these quantities to $A \in \mathbb{R}^{m \times k}$ and $B \in \mathbb{R}^{k \times n}$ on a $q \times q$ grid, assuming m, k, n are divisible by q.

(b) Decide what must change. [2 points]

Write $C_{i,j}$ as a sum of block products. Use this expression and your reasoning from Question 1 to decide whether rectangular blocks require changes to Cannon's initial alignment, shift directions, or number of stages. Explain why a square process grid can support rectangular matrix blocks.

Question 3: Dimensions That Do Not Divide Evenly (4 points)

Let A be 11×13 and B be 13×17, using nine processes. Consider adding zero rows and columns so that each matrix can be divided into equally sized blocks.

(a) Design the padding. [2 points]

Find the smallest padded dimensions of A and B and the resulting local block dimensions. Express your padding rule for general m, k, n and a $q \times q$ process grid. The padded matrices need not be square.

(b) Justify the result. [2 points]

Using the summation for a matrix-product entry, show why the added zeros preserve the desired result. Identify which rows and columns of the padded product belong in the output file, and explain why the remaining entries can be discarded.

Question 4: Performance and Scalability (3 points)

Model the runtime of Cannon's main loop for two $N \times N$ matrices on $p = q^2 > 1$ processes, where q divides N. Each process holds blocks of side length $b = N/q$.

Assume each of the q stages performs one local block multiplication costing $t_c b^3$, followed by two sequential block exchanges, each costing $t_s + t_w b^2$. All processes work concurrently, and computation and communication do not overlap. Ignore initial alignment, data distribution, result collection, and file I/O. Use $T_s = t_c N^3$ as the serial baseline; t_c, t_s, t_w are positive constants.

(a) Build the runtime model. [1 point]

Derive $T_p(N, p)$, separating computation time from communication time. Express your answer using N and p rather than b and q.

(b) Analyze scalability. [2 points]

Use your model to derive the efficiency $E = T_s/(pT_p)$. For fixed N, explain what happens to efficiency as p increases. How must N grow with p to maintain a fixed efficiency between 0 and 1? Express this requirement as an isoefficiency relation using serial work $W = N^3$.

Question 5: Implementation Analysis (3 points)

Describe the changes you made to the starter code and connect them to your reasoning in Questions 1–3. Include short code excerpts demonstrating the changes. Discuss one tradeoff in your implementation and how it affects memory use, computation, or communication. You may reference earlier answers instead of repeating an explanation.

8. Test Descriptions

The tests are grouped as follows (see `autograder.sh` for details):

- **Square Matrix Tests (0–4):** Multiplying square matrices of various sizes, e.g., $(2 \times 2)(2 \times 2)$, $(5 \times 5)(5 \times 5)$, $(4 \times 4)(4 \times 4)$, $(10 \times 10)(10 \times 10)$, and $(36 \times 36)(36 \times 36)$, with different process counts.
- **Divisible Matrix Tests (5–7):** Non-square matrices where dimensions are divisible by $\sqrt{p}$, e.g., $(2 \times 4)(4 \times 6)$, $(3 \times 6)(6 \times 9)$, and $(5 \times 10)(10 \times 25)$, with different process counts.
- **Prime Dimension Tests (8–10):** Matrices with prime dimensions, e.g., $(11 \times 13)(13 \times 17)$, $(13 \times 17)(17 \times 19)$, and $(17 \times 19)(19 \times 23)$. These dimensions require padding for the tested process grids with $q > 1$.

- **Timing Test (11):** Large matrix multiplication, $(1009 \times 1007)(1007 \times 1019)$, to check for speedup and performance.

Each test uses input files in `tests/test_X/in.txt` and compares your output to `tests/test_X/out.txt`. For Tests 0–7, we only use process counts such that each dimension of both A and B is divisible by $\sqrt{p}$.

9. Grading

Your final score is out of **30 points**: 18 for the written component and 12 for programming.

Component	Points
Written Question 1: Why square Cannon works	4
Written Question 2: Rectangular blocks	4
Written Question 3: Dimensions that do not divide evenly	4
Written Question 4: Performance and scalability	3
Written Question 5: Implementation analysis	3
Programming: Autograded correctness and performance	12
Total	**30**

Programming score (12 points): Your programming score is the score reported by the supplied autograder, out of 12. No scaling is applied.

Autograder deductions (12-point programming component):

- For each test that results in a timeout or runtime error: $-(9/53)$ points per failure, where 53 is the total number of tests.
- For each unit test that fails (incorrect output): $-(8/53)$ points per failure.
- For each speedup test that fails (Test 11, performance check): $-(3/8)$ points per failure, where 8 is the number of speedup checks.
- Your autograder score will not go below 0. Compilation failure results in 0 for the programming component. The autograder will run all tests and apply penalties as described above.

Written component (18 points): Questions 1–3 are worth 4 points each; Question 4 (performance and scalability) and Question 5 (implementation analysis) are worth 3 points each. Credit is based on your derivations and explanations, supported by the requested trace and code excerpts. The written component is graded separately from the programming component.

10. Submission

This assignment requires submission of two files to Canvas by **November 3, 11:59 PM**:

- `variable_cannon.c` — Your implementation of Cannon's algorithm, which can use a square number of processes $(1, 4, 9, 16, 25, 36, 49, 64)$ to perform matrix multiplication with arbitrary valid input dimensions.
- `report.pdf` — Your complete 18-point written submission: answers to Questions 1–5, including implementation analysis with code excerpts describing the changes you made to get your variable implementation to work properly.